УГНТУ • ЦЕНТР ТОП ИТ • 09.03.03 ПРИКЛАДНАЯ ИНФОРМАТИКА

Программирование и прототипирование ИТ-решений

Алгоритмы обработки коллекций — поиск, сортировка, map/filter, агрегация

ЛЕКЦИЯ 7

Ахметов Ильнур Вазирович

План лекции

1. Линейный и бинарный поиск: как быстро найти нужное.
2. Сортировка: sort(), sorted(), ключи и устойчивость.
3. Функции высшего порядка: map, filter, lambda.
4. Экстремумы и агрегация: max/min, sum, группировка, статистика.
5. Методы списка: полный набор и выбор под задачу.
6. Алгоритмические задачи аналитика: топ-N, аномалии, сводки.

В конце — вопросы для закрепления (по одному на слайд).

ПОВТОРЕНИЕ

Что мы уже умеем

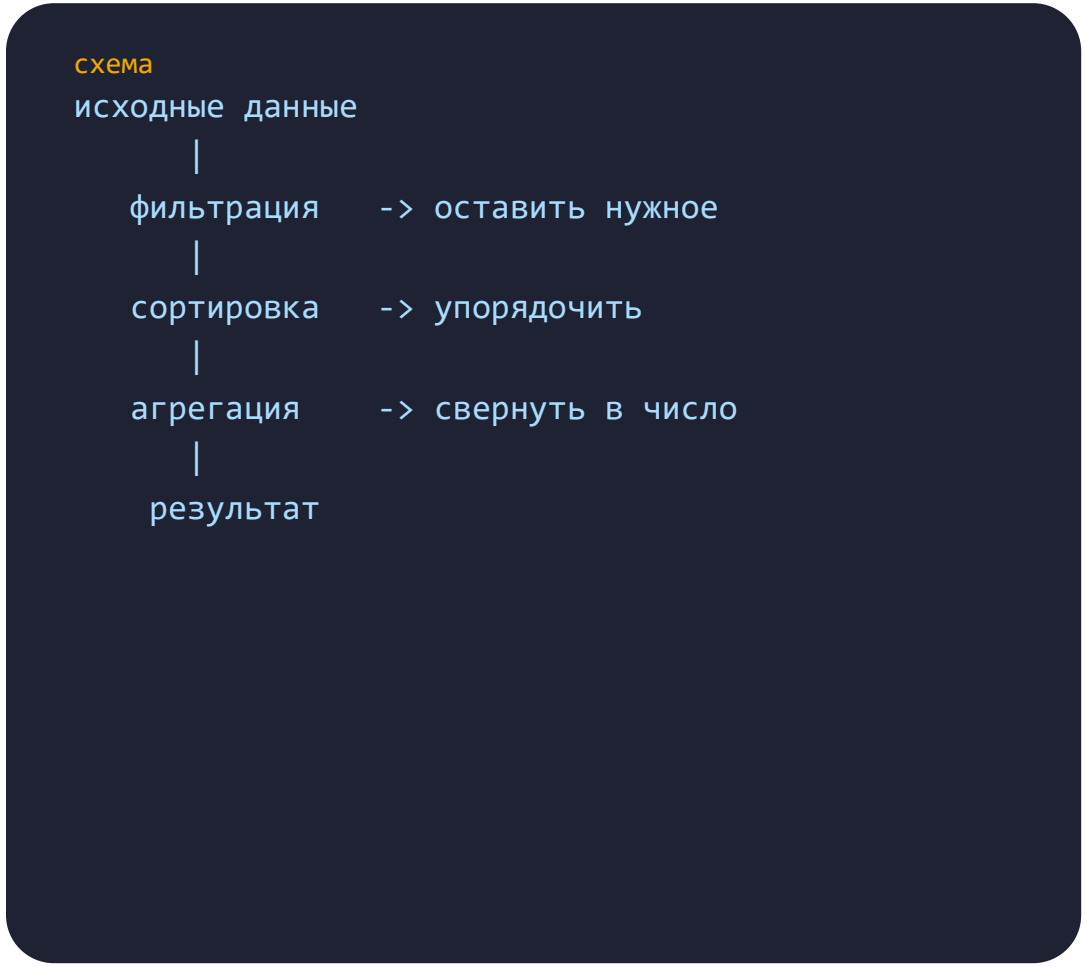
- Лекция 6: списки, кортежи, срезы, включения, генераторы.
- Мы научились ХРАНИТЬ данные в коллекциях.
- И даже строить новые коллекции включениями.
- Но остался главный вопрос аналитика:
- как НАЙТИ, ОТСОРТИРОВАТЬ и ОБОБЩИТЬ данные?
- Сегодня — алгоритмы обработки коллекций.

консоль Python

```
>>> debits = [85.4, 0.0, 12.7, 65.3]
>>> 65.3 in debits        # поиск
True
>>> sorted(debits)        # сортировка
[0.0, 12.7, 65.3, 85.4]
>>> max(debits)           # экстремум
85.4
>>> sum(debits)           # агрегация
163.4
```

Коллекция и алгоритм обработки

- Коллекция — набор данных: список, кортеж, словарь, множество.
- Алгоритм — конечная последовательность шагов для решения задачи.
- Алгоритм обработки коллекции отвечает на 4 вопроса:
- НАЙТИ элемент (поиск).
- УПОРЯДОЧИТЬ (сортировка).
- ПРЕОБРАЗОВАТЬ (map/filter).
- ОБОБЩИТЬ (экстремумы, сумма, группировка).



Сложность: почему это важно

- O(1) — одно действие, размер данных не важен.
- O(n) — линейный проход: время растёт пропорционально n.
- O(log n) — бинарный поиск: 1000 элементов → 10 шагов.
- O(n log n) — хорошая сортировка.
- O(n²) — вложенные циклы: опасно на больших данных.
- Аналитик выбирает алгоритм по размеру данных.

```
масштаб
n = 1 000
  O(1)       -> 1 операция
  O(log n)   -> ~10 операций
  O(n)       -> 1 000 операций
  O(n log n)-> ~10 000 операций
  O(n^2)     -> 1 000 000 операций

# 1 млн записей:
  O(n)       -> мгновенно
  O(n^2)     -> минуты
```

Линейный поиск: идём по одному

- Проверяем элементы по порядку, пока не найдём нужный.
- Работает на ЛЮБЫХ данных, сортировка не нужна.
- Сложность O(n): в худшем случае смотрим все n.
- Так работают in, index(), count() внутри.
- На 10 элементах — мгновенно; на 10 млн — заметно.

код

```
def linear_search(seq, target):
    """Возвращает индекс target или -1."""
    for i, x in enumerate(seq):
        if x == target:
            return i
    return -1

wells = ['K-12', 'K-13', 'K-14']
print(linear_search(wells, 'K-13'))   # 1
print(linear_search(wells, 'K-99'))   # -1
```

Встроенные: in, index, count

`x in seq` True/False — есть ли элемент (линейный проход).

`seq.index(x)` индекс первого вхождения, иначе ValueError.

`seq.count(x)` сколько раз встречается (тоже проход).

- Три вызова = три прохода. Если нужны все три — считайте один раз.
- Для больших данных частые in превращают O(n) в O(n·m).

консоль Python

```
>>> tags = ['нефть','газ','вода','газ']
>>> 'газ' in tags
True
>>> tags.index('вода')
2
>>> tags.count('газ')
2
>>> tags.index('сера')
ValueError: 'сера' is not in list
```

Поиск всех вхождений и по условию

- Часто нужен не первый, а ВСЕ подходящие элементы.
- Поиск по значению → включение с ==.
- Поиск по условию → включение с предикатом.
- any()/all() — быстрый ответ «есть ли хоть один / все ли».
- any/all останавливаются на первом решении — экономия времени.

код

```
debits = [85.4, 0.0, 12.7, 65.3, 0.0]

# все нули (остановленные)
stopped = [i for i, d in enumerate(debits) if d
== 0]
# [1, 4]

# есть ли хоть одна работающая?
print(any(d > 0 for d in debits))    # True

# все ли в норме?
print(all(0 <= d <= 100 for d in debits))  # True
```

Когда линейного поиска достаточно

- Данные небольшие (до нескольких тысяч) — линейный ок.
- Данные НЕ отсортированы — всё равно придётся пройти всё.
- Нужен ОДИН проход и сразу все показатели — считайте за раз.
- Ищем по сложному условию — линейный проход естественен.
- Если ищете много раз в одних данных — сортируйте и переходите
- к бинарному поиску или словарю (лекция 8).

правило

```
# плохо: 3 прохода по одному списку
if x in data: ...
idx = data.index(x)
n = data.count(x)

# лучше: один проход
found = idx = n = None
for i, v in enumerate(data):
    if v == x:
        if found is None:
            idx = i
        n = (n or 0) + 1
```

Идея: делим пополам

- Требование: данные ОТСОРТИРОВАНЫ.
- Смотрим в середину: если искомое меньше — идём влево,
- если больше — вправо. Каждый шаг отбрасывает ПОЛОВИНУ.
- 1000 элементов → 10 шагов; 1 000 000 → 20 шагов.
- Аналогия: поиск слова в бумажном словаре.
- Сложность O(log n) — принципиально быстрее O(n).

```
схема
[ 1  3  5  7  9 11 13 15 17 19 ]
              ^ середина = 11

ищем 15:  15 > 11 -> правая половина
          [ 13 15 17 19 ]
                   ^ 17
          15 < 17 -> левая
          [ 13 15 ]  -> нашли

# 10 элементов -> максимум 4 шага
```

Реализация: lo, hi, mid

- Два указателя: lo (начало) и hi (конец включительно).
- mid = (lo + hi) // 2 — середина.
- Условие цикла: while lo <= hi.
- Нашли — вернули mid; иначе сдвигаем границу.
- Не нашли — цикл закончился, возвращаем -1.

код

```
def binary_search(sorted_seq, target):
    """Индекс target в отсортированном списке или
-1."""
    lo, hi = 0, len(sorted_seq) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        value = sorted_seq[mid]
        if value == target:
            return mid
        elif value < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1
```

Трассировка по шагам

- Ищем 65.3 в отсортированном списке из 8 элементов.
- Каждый шаг сокращает диапазон вдвое — это видно в таблице.
- После 3 шагов остался один элемент — и он искомый.
- Для 8 элементов максимум 3–4 шага (log2 8 = 3).
- Умение трассировать = умение отлаживать алгоритм.

```
трассировка
data = [0.0, 12.7, 45.1, 52.0, 65.3, 70.0, 85.4,
90.0]
ищем 65.3

шаг 1: lo=0 hi=7 mid=3 -> 52.0 < 65.3 -> lo=4
шаг 2: lo=4 hi=7 mid=5 -> 70.0 > 65.3 -> hi=4
шаг 3: lo=4 hi=4 mid=4 -> 65.3 == 65.3 -> найден!

итого: 3 сравнения вместо 8 (линейный поиск)
```

Готовое решение: модуль bisect

- В стандартной библиотеке есть bisect — не пишите велосипед.

bisect_left(a, x) позиция вставки слева (первое >= x).

bisect_right(a, x) позиция вставки справа (первое > x).

insort(a, x) вставить с сохранением порядка.

- Разница left/right важна при дубликатах: она задаёт границу.

консоль Python

```
>>> from bisect import bisect_left, insort
>>> data = [10, 20, 30, 40]
>>> bisect_left(data, 25)
2                  # вставить перед 30
>>> bisect_left(data, 30)
2                  # первое вхождение 30
>>> bisect_right(data, 30)
3                  # сразу после 30
>>> insort(data, 25)
>>> data
[10, 20, 25, 30, 40]
```

Типичные ошибки

- Забыли отсортировать данные — поиск молча врёт.
- while lo < hi вместо <= — теряется последний элемент.
- hi = mid вместо mid - 1 — бесконечный цикл.
- lo = mid вместо mid + 1 — тоже зависание.
- Сравнение строк с числами — TypeError.
- Всегда проверяйте алгоритм на краях: первый, последний, отсутствующий.

проверка краёв

```
# эти случаи обязательны в тестах:
print(binary_search([], 5))           # -1
print(binary_search([7], 7))          # 0
print(binary_search([7], 5))          # -1
print(binary_search([1,3,5], 1))      # 0
print(binary_search([1,3,5], 5))      # 2
print(binary_search([1,3,5], 4))      # -1

# на неотсортированных данных ответ не определён!
```

Что выбрать: линейный или бинарный

Линейный — данные не отсортированы, ищем 1–2 раза, n мало.

Бинарный — данные отсортированы, ищем многократно, n велико.

- Сортировка стоит O(n log n) один раз и окупается.
- Если поисков много — сортировка + бинарный выигрывает.
- Если поиск один — линейный дешевле (не надо сортировать).
- Для частых поисков по ключу лучше словарь: O(1) (лекция 8).

сравнение

```
данные = 10 000 элементов, 1 000 поисков

линейный:  1000 * 10000 = 10 000 000 операций
бинарный:  сортировка ~130 000 + 1000*14 = ~144
000

# выигрыш ~70 раз
# но если поиск ОДИН:
#   линейный: 10 000
#   бинарный: 130 000 (сортировка) + 14
```

Зачем сортировать

- Сортировка — упорядочивание элементов по правилу.
- Открывает бинарный поиск (O(log n)).
- Даёт рейтинги и топ-N (самые большие/маленькие).
- Делает отчёты и таблицы читаемыми для человека.
- Помогает находить дубликаты и аномалии (они «всплывают» рядом).
- Сортировка — базовая операция аналитика.

код

```
debits = [85.4, 0.0, 12.7, 65.3]

print(sorted(debits))
# [0.0, 12.7, 65.3, 85.4]

# топ-2 лучших скважин:
print(sorted(debits, reverse=True)[:2])
# [85.4, 65.3]

# бинарный поиск теперь возможен
```

sorted() против sort()

sorted(seq) возвращает НОВЫЙ список, оригинал не меняет.

seq.sort() сортирует НА МЕСТЕ, возвращает None.

- Оба принимают key= и reverse=.
- Помните ловушку: d = d.sort() → d станет None.
- Для кортежа и строки доступен только sorted() — они неизменяемы.

консоль Python

```
>>> d = [3, 1, 2]
>>> sorted(d)
[1, 2, 3]
>>> d
[3, 1, 2]        # не изменился

>>> d.sort()
>>> d
[1, 2, 3]        # изменился

>>> t = (3, 1, 2)
>>> sorted(t)
[1, 2, 3]        # кортеж -> список
```

Ключ сортировки: key=

- key — функция, возвращающая «вес» элемента для сравнения.
- Сравниваются не сами элементы, а результаты key(элемент).
- Чаще всего key=lambda x: ... — короткая безымянная функция.
- По умолчанию сравниваются сами элементы.
- key не меняет данные — только задаёт правило порядка.

код

```
wells = [
    {"id": "К-12", "дебит": 85.4},
    {"id": "К-13", "дебит": 12.7},
    {"id": "К-14", "дебит": 65.3},
]

by_debit = sorted(wells, key=lambda w:
w['дебит'])
# 12.7 -> 65.3 -> 85.4

# по убыванию:
top = sorted(wells, key=lambda w: w['дебит'],
             reverse=True)
```

Несколько ключей и строки

- Кортеж в key даёт сортировку по НЕСКОЛЬКИМ полям.
- Порядок сравнения — слева направо, как в кортеже.
- Отрицательный вес (-x) разворачивает отдельное поле.
- Строки сортируются посимвольно (лексикографически).
- Регистр влияет: 'Я' < 'a' в Unicode — используйте key=str.lower.

код

```
# сначала по группе, потом по дебиту
rows = [('А', 85.4), ('Б', 12.7), ('А', 65.3)]
print(sorted(rows, key=lambda r: (r[0], r[1])))
# [('А', 65.3), ('А', 85.4), ('Б', 12.7)]

# по группе возрастанию, дебиту убыванию:
print(sorted(rows, key=lambda r: (r[0], -r[1])))

# строки без учёта регистра:
names = ['Уфа', 'казань', 'Тюмень']
print(sorted(names, key=str.lower))
```

itemgetter и attrgetter

- operator.itemgetter(i) — берёт i-й элемент (для кортежей/списков).
- operator.itemgetter(k) — берёт значение по ключу (для словарей).
- attrgetter('field') — берёт атрибут объекта.
- Работает быстрее lambda и читается короче.
- Для нескольких полей: itemgetter(0, 1).

код

```
from operator import itemgetter

rows = [('A', 85.4), ('Б', 12.7)]
print(sorted(rows, key=itemgetter(1)))

wells = [{'id': 'K-12', 'W': 0.24},
         {'id': 'K-13', 'W': 0.91}]
print(sorted(wells, key=itemgetter('W')))

# несколько полей:
print(sorted(rows, key=itemgetter(0, 1)))
```

Устойчивость сортировки

- Сортировка в Python УСТОЙЧИВАЯ (stable).
- Элементы с равным ключом сохраняют исходный порядок.
- Это позволяет сортировать «в два прохода»:
- сначала по второстепенному полю,
- потом по главному — второстепенный сохранится.
- Практика: сначала по дате, потом по сумме — внутри сумм дата останется.

код

```
rows = [('2024-01-05', 100),
        ('2024-01-03', 100),
        ('2024-01-04', 50)]

# 1) по дате
rows.sort(key=lambda r: r[0])
# 2) по сумме (устойчиво!)
rows.sort(key=lambda r: r[1])

# внутри равных сумм дата сохранилась
# -> [('2024-01-04',50),
#     ('2024-01-03',100),
#     ('2024-01-05',100)]
```

Как это работает внутри и ловушки

- Python использует Timsort — гибрид сортировки вставками и слиянием.
- Сложность O(n log n), на почти отсортированных данных — быстрее.
- Ловушка: сравнение несравнимых типов (число с строкой) → TypeError.
- Ловушка: None в данных → TypeError при сравнении.
- Решение: key, приводящий к одному типу, или фильтрация None.

ловушки

```
# TypeError: '<' not supported
sorted([1, '2', 3])

# TypeError: None в данных
sorted([5, None, 3])

# решение 1: фильтр
clean = [x for x in data if x is not None]

# решение 2: ключ с подстановкой
sorted(data, key=lambda x: (x is None, x))
```

Функция высшего порядка

- В Python функции — такие же объекты, как числа.
- Их можно передавать в другие функции как аргумент.
- Функция высшего порядка (HOF) — принимает функцию
- или возвращает функцию.
- map, filter, sorted(key=...), max(key=...) — все HOF.
- Это позволяет отделить «что делаем» от «как обходим».

идея

```
def apply(func, data):
    return [func(x) for x in data]

def double(x):
    return x * 2

print(apply(double, [1, 2, 3]))
# [2, 4, 6]

# func — это ПАРАМЕТР, а не вызов
```

map: применить ко всем

- map(функция, последовательность) — применяет функцию к каждому.
- Возвращает ленивый итератор (не список!).
- Чтобы получить список — оберните в list().
- Эквивалент включения [f(x) for x in seq].
- Полезен, когда функция уже есть и не хочется писать lambda.

консоль Python

```
>>> map(str, [1, 2, 3])
<map object at 0x...>
>>> list(map(str, [1, 2, 3]))
['1', '2', '3']

>>> def f2c(f):
...     return (f - 32) * 5 / 9
>>> list(map(f2c, [32, 212]))
[0.0, 100.0]
```

map на нескольких последовательностях

- map может принимать НЕСКОЛЬКО последовательностей.
- Работает как zip: берёт элементы попарно.
- Останавливается на самой короткой.
- Идеально для поэлементных операций: сумма, произведение, разность.
- Аналог: [f(a, b) for a, b in zip(A, B)].

консоль Python

```
>>> from operator import add, mul
>>> a = [1, 2, 3]
>>> b = [10, 20, 30]
>>> list(map(add, a, b))
[11, 22, 33]
>>> list(map(mul, a, b))
[10, 40, 90]

>>> list(map(add, [1, 2], [10]))
[11]            # короткая остановила
```

filter: отбор по предикату

- filter(предикат, последовательность) — оставляет элементы,
- для которых предикат вернул True.
- Тоже возвращает итератор — нужен list().
- Эквивалент [x for x in seq if pred(x)].
- Предикат — функция, возвращающая bool.
- filter(None, seq) отбрасывает «ложные» значения (0, '', None).

консоль Python

```
>>> def is_active(d):
...     return d > 0
>>> debits = [85.4, 0.0, 12.7, 0.0]
>>> list(filter(is_active, debits))
[85.4, 12.7]

>>> list(filter(None, [0, 1, '', 'x', None]))
[1, 'x']
```

map/filter или включения?

- Функционально map/filter и включения делают одно и то же.
- Включения чаще читаемее и понятнее новичку.
- map/filter удобны, когда функция уже определена по имени.
- Оба ленивы только в варианте-генераторе; list() материализует.
- Правило команды важнее «моды»: выберите один стиль.
- В аналитике чаще берут включения за читаемость.

эквиваленты

```
# map
list(map(len, ['a', 'bb']))      # [1, 2]
[len(s) for s in ['a', 'bb']]    # то же

# filter
list(filter(bool, data))
[x for x in data if x]

# комбинация: map+filter = включение
list(map(len, filter(bool, data)))
[len(x) for x in data if x]
```

lambda в HOF

- lambda — безымянная функция из одного выражения.
- Синтаксис: lambda параметры: выражение.
- Удобна как «одноразовый» аргумент в key/map/filter.
- Не может содержать операторы (только выражение).
- Для сложной логики лучше обычная def — читаемее и отлаживаемее.

консоль Python

```
>>> sq = lambda x: x ** 2
>>> sq(5)
25

>>> list(map(lambda x: x * 10, [1, 2]))
[10, 20]

>>> rows = [('A', 2), ('Б', 1)]
>>> sorted(rows, key=lambda r: r[1])
[('Б', 1), ('A', 2)]
```

Агрегация через reduce и цепочки

- reduce(функция, последовательность) сворачивает к одному значению.
- Живёт в functools (в Python 3 не встроен).
- sum() — частный случай reduce со сложением.
- Цепочка map->filter->sum — типовой конвейер обработки.
- Для агрегаций чаще есть готовые встроенные (sum, max, min, any).

код

```
from functools import reduce

nums = [1, 2, 3, 4]
print(reduce(lambda a, b: a * b, nums))  # 24
print(sum(nums))                         # 10

# конвейер: взять активные, удвоить, сложить
debits = [85.4, 0.0, 12.7]
active = filter(lambda d: d > 0, debits)
doubled = map(lambda d: d * 2, active)
print(sum(doubled))                      # 196.2
```

max и min

- max(seq), min(seq) — наибольший и наименьший элемент.
- Принимают key=, как и сортировка.
- default= спасает от ошибки на пустой коллекции.
- На словаре max вернёт ключ с максимальным значением
- сравнения ключей — обычно нужен key.
- Работают за один проход O(n).

консоль Python

```
>>> debits = [85.4, 12.7, 65.3]
>>> max(debits), min(debits)
(85.4, 12.7)

>>> max([], default=0)
0

>>> wells = [{'id':'K-12','W':0.24},
...          {'id':'K-13','W':0.91}]
>>> max(wells, key=lambda w: w['W'])['id']
'K-13'
```

Поиск максимума вручную

- Алгоритм: берём первый элемент как «лучший»,
- проходим остальные и обновляем лучший при необходимости.
- Один проход, O(n), без сортировки.
- Понимание важно: max() внутри делает то же самое.
- На интервью и в жизни просят «найти без встроенных».
- Считайте и индекс, чтобы получить позицию максимума.

код

```
def find_max(seq):
    """Возвращает (значение, индекс)
максимума."""
    best, best_i = seq[0], 0
    for i in range(1, len(seq)):
        if seq[i] > best:
            best, best_i = seq[i], i
    return best, best_i

print(find_max([85.4, 12.7, 65.3]))
# (85.4, 0)
```

argmax: индекс максимума

- Часто нужен не только максимум, но и ГДЕ он (индекс/ключ).

`seq.index(max(seq))` — два прохода, но коротко.

`max(enumerate(seq), key=...)` — один проход, возвращает пару.

- Для словаря — max(d, key=d.get) даёт ключ максимума.
- Для рейтинга — max по key=lambda.

консоль Python

```
>>> debits = [12.7, 85.4, 65.3]
>>> debits.index(max(debits))
1

>>> pairs = list(enumerate(debits))
>>> max(pairs, key=lambda p: p[1])
(1, 85.4)

>>> d = {'K-12': 12.7, 'K-13': 85.4}
>>> max(d, key=d.get)
'K-13'
```

Топ-N: не один, а несколько

`sorted(seq, reverse=True)[:n]` простая сортировка целиком, O(n log n).

`heapq.nlargest(n, seq)` эффективнее при малом n на больших данных.

`heapq.nsmallest(n, seq, key=)` то же для минимумов.

- nlargest/nsmallest выгодны, когда n << len(seq).
- Для отчётов «топ-10 скважин» — обе подходят, но различайтесь по памяти.

код

```
from heapq import nlargest, nsmallest

debits = [85.4, 12.7, 65.3, 40.0, 0.0]

print(sorted(debits, reverse=True)[:2])
# [85.4, 65.3]

print(nlargest(2, debits))
# [85.4, 65.3]

print(nsmallest(2, debits))
# [0.0, 12.7]
```

Агрегация: sum, len, среднее

- Агрегация — свёртка коллекции в одно число.
- sum(seq) — сумма; len(seq) — количество.
- Среднее = sum(seq) / len(seq) — но помните о пустой коллекции!
- Пустой список → ZeroDivisionError при делении на len.
- Для среднего есть statistics.mean с той же проблемой —
- поэтому проверка на пустоту обязательна.

код

```
debits = [85.4, 12.7, 65.3]

total = sum(debits)          # 163.4
n = len(debits)              # 3
avg = total / n              # 54.47

# безопасно:
def safe_mean(seq):
    return sum(seq) / len(seq) if seq else 0.0

print(safe_mean([]))         # 0.0
```

Группировка: dict и Counter

- Группировка: разбить данные на группы по признаку и обобщить.
- Вручную: словарь {группа: [элементы]} через setdefault.
- Counter — готовый счётчик вхождений (collections).
- Counter.most_common(n) — топ по частоте.
- Это ручной аналог GROUP BY из SQL.

код

```
from collections import Counter, defaultdict

statuses = ['ok','err','ok','ok','err']
print(Counter(statuses))
# Counter({'ok': 3, 'err': 2})
print(Counter(statuses).most_common(1))
# [('ok', 3)]

groups = defaultdict(list)
for item in data:
    groups[item['куст']].append(item['дебит'])
```

Статистика: медиана и мода

- Модуль statistics: mean, median, mode, stdev.
- Среднее чувствительно к выбросам, медиана — устойчива.
- Медиана = середина отсортированного ряда.
- Мода = самое частое значение (аналог most_common).
- Для анализа показателей часто берут медиану, а не среднее.

консоль Python

```
>>> from statistics import mean, median, mode
>>> data = [10, 12, 13, 14, 1000]
>>> mean(data)
209.8            # выброс тянет вверх
>>> median(data)
13               # типичное значение
>>> mode([1, 2, 2, 3])
2
```

Полный обзор методов списка

append(x) добавить в конец

insert(i, x) вставить по индексу

extend(seq) добавить все элементы

pop([i]) удалить и вернуть

remove(x) удалить первое вхождение

clear() очистить

index(x) индекс первого вхождения

count(x) число вхождений

```
ещё методы
sort(key=, reverse=)   # на месте, -> None
reverse()              # развернуть на месте
copy()                 # поверхностная копия

# встроенные функции (не методы):
len, sum, min, max, sorted,
any, all, reversed, enumerate, zip

# ВСЕ методы-мутаторы возвращают None!
```

Добавление и удаление

- append/insert/extend — три способа добавить (см. лекцию 6).
- pop/remove/clear — три способа удалить.
- pop возвращает удалённый элемент, remove — нет.
- Все мутаторы возвращают None — не присваивайте результат себе.
- Удаление в цикле по тому же списку — классическая ошибка.

консоль Python

```
>>> q = ['a', 'b', 'c']
>>> q.append('d')
>>> q.pop()
'd'
>>> q.insert(0, 'z')
>>> q
['z', 'a', 'b', 'c']
>>> q.remove('a')
>>> q
['z', 'b', 'c']
```

Поиск и счёт

- index(x) — индекс первого вхождения, ValueError если нет.
- count(x) — сколько раз встречается.
- in — проверка наличия без исключения.
- Все три — линейные проходы O(n).
- Если нужны оба (индекс и количество) — посчитайте за один проход.

консоль Python

```
>>> w = ['K-12', 'K-13', 'K-12']
>>> w.index('K-12')
0
>>> w.count('K-12')
2
>>> 'K-14' in w
False

>>> w.index('K-14')
ValueError: 'K-14' is not in list
```

Сортировка, разворот, копия

- sort() — на месте, возвращает None, принимает key и reverse.
- sorted() — новый список, оригинал не трогает.
- reverse() — разворот на месте; reversed() — итератор.
- copy() — поверхностная копия; для вложенных нужен deepcopy.
- Ловушка: b = a — ссылка, a.copy() — копия.

консоль Python

```
>>> d = [3, 1, 2]
>>> d.sort()
>>> d
[1, 2, 3]
>>> d.reverse()
>>> d
[3, 2, 1]

>>> s = sorted(d)      # новый список
>>> d.copy() is d
False
```

Какой метод выбрать: чек-лист

- Добавить один элемент → append.
- Добавить много → extend (или +).
- Вставить в середину → insert.
- Удалить по значению → remove; по позиции → pop.
- Найти позицию → index; посчитать → count.
- Упорядочить → sort (на месте) или sorted (копия).
- Удалить много → включение или del по срезу.

шпаргалка

```
# чего НЕТ у списка:
lst.first()    -> lst[0]
lst.last()     -> lst[-1]
lst.contains() -> x in lst
lst.max()      -> max(lst)
lst.sum()      -> sum(lst)

# методы vs функции:
lst.sort()  <>  sorted(lst)
lst.reverse() <> reversed(lst)
```

Топ-N по показателю

- Типовая задача: «покажи 5 скважин с максимальным дебитом».
- Шаги: отфильтровать невалидные → отсортировать → взять срез.
- Сортировка по key, reverse=True, затем [:n].
- Для больших данных — nlargest.
- Не забудьте про скважины с нулевым дебитом (их не должно быть в топе).

код

```
wells = [
    {"id": "K-12", "q": 85.4},
    {"id": "K-13", "q": 0.0},
    {"id": "K-14", "q": 120.5},
    {"id": "K-15", "q": 65.3},
]
active = [w for w in wells if w['q'] > 0]
top2 = sorted(active, key=lambda w: w['q'],
              reverse=True)[:2]
# K-14 (120.5), K-12 (85.4)
```

Фильтрация и агрегация

- Ещё одна типовая связка: отобрать подмножество и обобщить.
- Фильтр — включение или filter().
- Агрегация — sum/len/max/min или statistics.
- Комбинируйте: средний дебит только по активным скважинам.
- Проверяйте пустое подмножество — иначе деление на ноль.

код

```
debits = [85.4, 0.0, 12.7, 65.3, 0.0]

active = [d for d in debits if d > 0]

if active:
    avg = sum(active) / len(active)
    print(f'Средний по активным: {avg:.2f}')
else:
    print('Нет работающих скважин')

# Средний по активным: 54.47
```

Поиск аномалий и выбросов

- Аномалия — значение, резко отличающееся от остальных.
- Простой способ: порог (выше/ниже границы).
- Устойчивее: сравнение с медианой и разбросом.
- Один линейный проход находит все аномалии.
- Выбросы нельзя молча отбрасывать — их надо показать аналитику.

код

```
from statistics import median

data = [82, 85, 84, 86, 1400, 83]
med = median(data)          # 84.5

# простой порог:
outliers = [x for x in data if abs(x - med) > 3 *
med]

# ещё вариант — по порогу сверху:
high = [x for x in data if x > 500]

print(high)      # [1400]
```

Слияние двух источников (два указателя)

- Задача: объединить два отсортированных списка.
- Наивно: склеить и отсортировать — O(n log n).
- Два указателя: O(n + m) за один проход.
- Классика алгоритмических собеседований и ETL-задач.
- Так же сливают два источника данных по ключу.

код

```
def merge_sorted(a, b):
    i = j = 0
    out = []
    while i < len(a) and j < len(b):
        if a[i] <= b[j]:
            out.append(a[i]); i += 1
        else:
            out.append(b[j]); j += 1
    out.extend(a[i:]); out.extend(b[j:])
    return out

print(merge_sorted([1, 4, 7], [2, 3, 9]))
# [1, 2, 3, 4, 7, 9]
```

Сводная таблица по группам

- Аналог сводной таблицы Excel и GROUP BY в SQL.
- 1) группируем по признаку, 2) агрегируем каждую группу.
- Реализация: defaultdict(list), затем sum/len/max.
- Результат — словарь {группа: показатель}.
- Это мост к pandas: там то же самое — groupby().agg().

код

```
from collections import defaultdict

wells = [('Куст 1', 85.4), ('Куст 2', 12.7),
         ('Куст 1', 65.3), ('Куст 2', 40.0)]

groups = defaultdict(list)
for cluster, q in wells:
    groups[cluster].append(q)

summary = {c: round(sum(v)/len(v), 2)
           for c, v in groups.items()}
# {'Куст 1': 75.35, 'Куст 2': 26.35}
```

Типичные ошибки и ловушки

`d = d.sort()` — sort() возвращает None — список уничтожается. Используйте sorted().

Бинарный поиск без сортировки — на неотсортированных данных ответ не определён.

`while lo < hi` — теряется последний элемент; правильно lo <= hi.

Среднее без проверки пустоты — ZeroDivisionError: sum(seq)/len(seq) при пустом списке.

max() вместо max(key=…) — для словаря вернёт ключ, а не значение.

Удаление в цикле — for x in lst: lst.remove(x) — пропуски; создавайте новый список.

Итоги лекции

1. Линейный поиск O(n) — универсален; бинарный O(log n) — только на отсортированных.
2. sorted() возвращает новый список, sort() меняет на месте и возвращает None.
3. key= задаёт правило сортировки; кортеж в key — несколько полей.
4. map и filter — ленивые итераторы; включения — тот же результат, читаемее.
5. max/min за один проход; top-N — nlargest или срез после сортировки.
6. Агрегация: sum, len, среднее с проверкой пустоты; группировка через dict.
7. Аналитик решает: топ-N, фильтр+агрегация, аномалии, сводка по группам.

Вопрос 1 из 6

Почему бинарный поиск требует отсортированных данных и какова его сложность?

Вопрос 2 из 6

Чем sorted() отличается от sort()? Что вернёт lst.sort()?

Вопрос 3 из 6

Что произойдёт, если в max(debits) передать пустой список? Как это исправить?

Вопрос 4 из 6

Чем map/filter отличаются от списковых включений? Что выбрать и почему?

Вопрос 5 из 6

Как найти индекс максимального элемента за один проход по списку?

Вопрос 6 из 6

Как сгруппировать данные по признаку и посчитать среднее по каждой группе?